

956 lines (794 loc) · 55.9 KB

PreviewCodeBlame

Raw

MGraph-AI Service Base: Production-Ready Microservice Template

Technical Briefing Document

Version: 1.0
Last Updated: January 2026
Audience: CTOs, Engineering Leaders, Platform Engineers

Executive Summary

MGraph-AI Service Base is an opinionated microservice template that provides **complete production infrastructure before a single line of business logic is written**. It eliminates the gap between "I have an idea" and "I have a production-ready service" by front-loading all operational concerns into a reusable, testable foundation.

This document explains the business rationale, technical architecture, and operational workflows that make this approach effective at scale.

Part 1: The Business Case

The Hidden Cost of "We'll Productionize Later"

Most engineering teams follow a familiar pattern: build the feature first, worry about deployment later. This creates a predictable set of problems:

Week 1-4: Build feature in local environment
Week 5: "How do we deploy this?"
Week 6-8: Build CI/CD pipeline from scratch
Week 9: "Tests are failing in CI but pass locally"
Week 10: "We need staging environments"
Week 11-12: Debug environment-specific issues
Week 13: Finally in production



The actual feature took 4 weeks. Infrastructure took 9 weeks.

This pattern repeats for every new service, with slight variations. Teams accumulate technical debt not in their application code, but in inconsistent deployment processes, missing test coverage, and environment drift.

The "Infrastructure-First" Alternative

MGraph-AI Service Base inverts this model:

Hour 0: Run bootstrap script
Hour 0.5: CI/CD pipeline active, tests passing, Lambda deployment ready
Hour 1+: Write business logic with confidence



When the first line of business code is written, the following already exists:

Capability	Status	Notes
Unit test framework	✅ Active	Tests run on every commit
FastAPI scaffold	✅ Running	Health checks, versioning endpoints
Local development server	✅ Works	Hot reload, mirrors production
CI/CD pipeline	✅ Configured	GitHub Actions, 3-stage deployment
Automated versioning	✅ Active	Every commit tagged, files synced
AWS Lambda deployment	✅ Ready	Add credentials and deploy
API authentication	✅ Implemented	API key middleware configured
PyPI publishing	✅ Available	Package structure ready

Quantified Benefits

For Individual Services:

- 2-4 weeks saved per service on infrastructure setup
- Zero "works on my machine" deployment issues
- Guaranteed test coverage from day one

For Engineering Organizations:

- Consistent operational patterns across all services

- New team members productive immediately (same structure everywhere)
- Reduced cognitive load when context-switching between services

For Release Management:

- Every deployment traceable to exact commit and version
- Instant rollback capability via tagged releases
- Clear audit trail for compliance requirements

Why Not GitHub Templates?

GitHub's template feature creates a one-time copy. This approach uses a **living base repository**:

GitHub Templates	Living Base Repository
Static snapshot at creation time	Can pull upstream improvements
No way to test the template itself	Base repo has its own test suite
Diverges immediately	Shared improvements benefit all services
Template bugs discovered per-project	Template bugs fixed once, propagated everywhere

The base repository is itself version-controlled, tested, and improved over time. Services created from it can selectively incorporate improvements.

Part 2: Technical Architecture

Repository Structure

MGraph-AI__Service__Base/

├─ mgraph_ai_service_base/

├─ fast_api/

├─ service/

└─ utils/

├─ __init__.py

├─ config.py

├─ version

├─ Base_Service_Fast_API.py

├─ lambda_handler.py

├─ routes/

├─ info/

├─ Service_Info.py

├─ schemas/

└─ Version.py

Application code

Package metadata

Service configuration

Single source of truth for version

API layer

FastAPI app setup

AWS Lambda entry point

Endpoint definitions

Business logic layer

Service metadata

Pydantic/Type_Safe models

Utilities

Version management

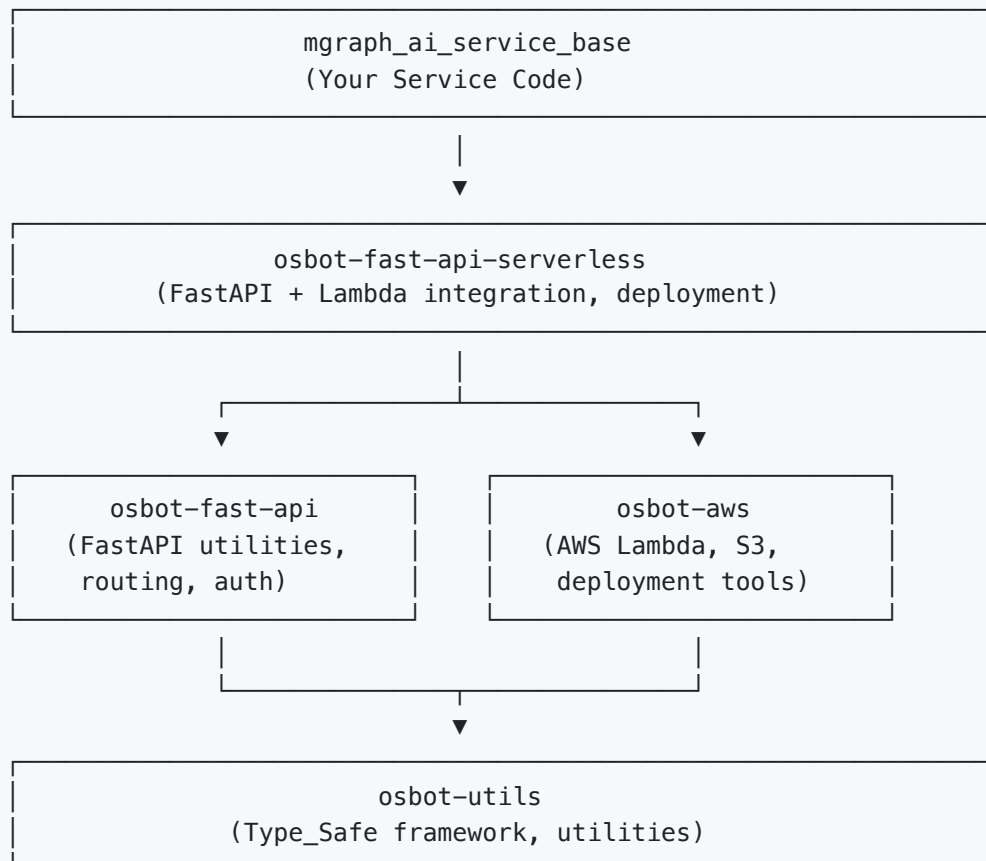
```

├── deploy/                                # Deployment utilities
│   └── Deploy__Service.py
├── tests/
│   ├── unit/                             # Unit tests (run in CI)
│   └── deploy_aws/                       # Deployment tests (run on deploy)
├── scripts/
│   ├── run-locally.sh                    # Local development server
│   └── setup-new-service-from-bare-github-repo.sh # Bootstrap script
├── .github/
│   ├── workflows/                       # CI/CD pipeline definitions
│   │   ├── ci-pipeline.yml              # Base workflow (reusable)
│   │   ├── ci-pipeline_dev.yml          # Dev branch triggers
│   │   ├── ci-pipeline_main.yml         # Main branch triggers
│   │   └── ci-pipeline_prod.yml         # Production (manual)
│   └── actions/
│       └── aws__deploy__lambda/         # Lambda deployment action
├── pyproject.toml                        # Python package definition
├── requirements-test.txt                 # Test dependencies
└── README.md                            # Documentation (version badge auto-
updated)

```

Dependency Architecture

The service builds on a layered utility framework:



Key Design Decision: Heavy use of `Type_Safe` base class provides runtime type checking, automatic serialization, and consistent object patterns across all services.

Lambda Handler Architecture

The Lambda entry point (`lambda_handler.py`) implements a sophisticated zero-dependency bootstrap pattern that avoids the need for Lambda Layers while providing better flexibility and often better performance.

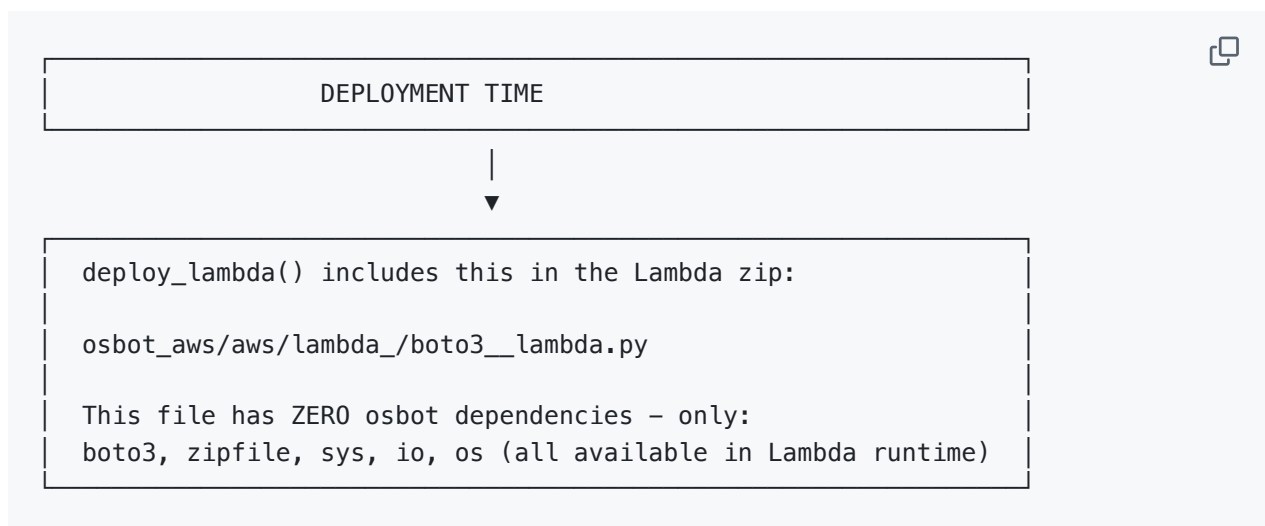
The Bootstrap Problem

The challenge: How do you load dependencies (like `osbot-fast-api-serverless`) in a Lambda function without including them in the deployment package or using Layers?

The solution: A single-file bootstrap loader that only depends on what Lambda already provides.

Deployment Time: Injecting the Bootstrap File

When `deploy_lambda()` runs, it calls `add_file__boto3__lambda()` :

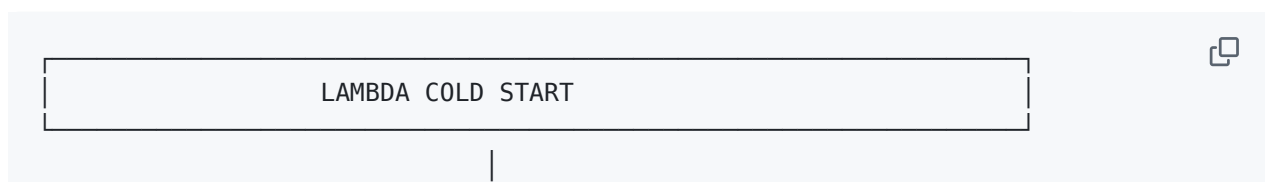


The `boto3__lambda.py` file contains just two functions:

```
def load_dependency(package_name):
    # 1. Get S3 bucket: {account_id}--osbot-lambdas--{region}
    # 2. Download: lambdas-dependencies/{package}.zip
    # 3. Extract to: /tmp/lambdas-dependencies/{package}/
    # 4. Append to sys.path

def load_dependencies(packages):
    return [load_dependency(p) for p in packages]
```

Runtime: The Cold Start Sequence



▼

```
from osbot_aws.aws.lambda_.boto3__lambda import load_dependencies
```

This import works in BOTH environments:

- Locally: resolves to full osbot-aws package (installed)
- In Lambda: resolves to the single injected file

↓

```
load_dependencies(['osbot-fast-api-serverless==v1.2.0'])
```

Downloads from S3 → Extracts to /tmp → Adds to sys.path

↓

THE PROBLEM: Python module resolution conflict

sys.modules['osbot_aws'] now points to the STUB (single file)

But we just downloaded the REAL osbot_aws to /tmp/...
Python won't load it because it thinks osbot_aws is already loaded!

↓

```
clear_osbot_modules()
```

```
for module in sys.modules:  
    if module.startswith('osbot_aws'):  
        del sys.modules[module] # Remove the stub reference
```

Forces Python to RE-RESOLVE imports from the new sys.path
which now includes /tmp/lambda-dependencies/...

↓

```
Initialize FastAPI app with full dependency tree available  
Base__Service__Fast_API().setup()
```

↓

```
Export: handler, app, run
```

Why S3 Instead of Lambda Layers?

Lambda Layers	S3 Dynamic Loading
Requires layer version management	Just update the zip in S3

Lambda Layers	S3 Dynamic Loading
Shared across functions (can cause conflicts)	Per-service dependencies
5 layer limit per function	No limit
Deployment coupling	Update dependencies without redeploying
	Surprisingly faster in many cases

The S3 approach is essentially: **GET zip** → **Extract** → **Set path**. The overhead is minimal, and the flexibility gains are significant.

Why the Same Import Path Works Everywhere

This is the elegant part:

```
from osbot_aws.aws.lambda_.boto3__lambda import load_dependencies
```



- **Locally/Tests:** Full `osbot-aws` package is installed → import finds real package
- **In Lambda:** Single file exists at `osbot_aws/aws/lambda_/boto3__lambda.py` → import finds stub

No conditional imports. No environment detection. The same code runs everywhere.

Part 3: The Git Tagging Workflow

The Version Synchronization Problem

Without automation, version information drifts across files:

```
# The nightmare scenario
$ git tag           # Shows v1.2.3
$ cat version       # Shows v1.2.4
$ cat pyproject.toml # Shows version = "1.2.2"
$ cat README.md     # Shows badge for v1.2.5
```

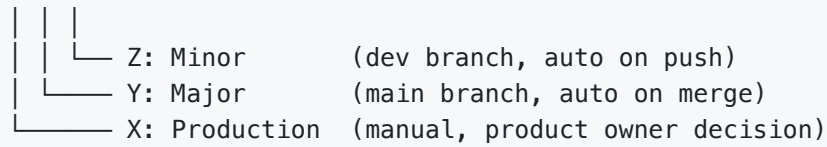


Deployment pipelines fail. Support can't identify versions. Rollbacks become guesswork.

Automated Version Management

The Three Release Types

The versioning system recognizes three distinct release types, each tied to a specific branch and quality gate:



Release Type	Version Pattern	Branch	Trigger	Purpose
minor	v1.2.3 → v1.2.4	dev	Auto on push	Day-to-day development, bug fixes, features in progress
major	v1.2.x → v1.3.0	main	Auto on merge	QA-ready releases, feature-complete milestones
production	v1.x.x → v2.0.0	manual	Product owner decision	Production deployment, customer-facing release

Note: This workflow intentionally has no "patch" or "hotfix" concept. Even urgent security fixes flow through the standard dev → main → production pipeline. This maintains the integrity of the testing and review process.

Code Quality Expectations

The core principle: If tests pass, code is ready for production.

This is not aspirational—it's enforced by the workflow design:



DEV BRANCH

- Code should ALWAYS be in solid, shippable state
- All tests MUST pass (tagging depends on test success)
- Should be OK to merge into main at any time
- Work-in-progress or broken code belongs in feature branches
OFF dev, not in dev itself

```
merge to main
```

▼

MAIN BRANCH

- ALWAYS in a state that can ship to production
- Every major release is a candidate for production
- The decision to release is BUSINESS, not technical

product owner approval

▼

PRODUCTION RELEASE

- Exact same code as the major release ($X == Y$)
- Only the tag and release notes change
- This is a BUSINESS decision, not a code decision

This approach encourages healthy development practices:

- **Developing new features side-by-side** with existing functionality
- **Feature toggles** to ship incomplete features safely
- **Limited breaking changes** because dev is always near-shippable
- **Frequent integration** rather than long-lived feature branches

Release Patterns in Practice

For rapidly evolving projects: Major releases may go directly to production. The high velocity means QA-tested code (main) is often immediately promoted.

For mature/stable projects: Only production releases become official customer-facing versions. It's perfectly normal for version numbers to grow large—look at Chrome's version numbers as an example.

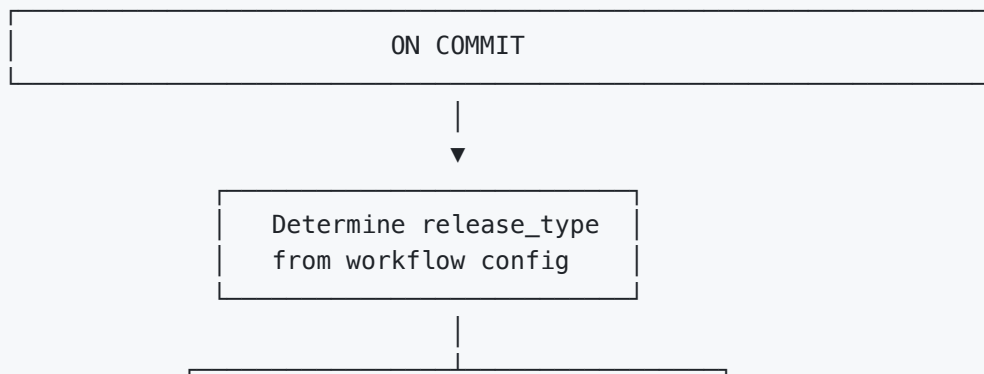
Rapidly Evolving:	<code>v0.47.0 (main) → immediately promoted → v1.0.0 (prod)</code>
	<code>v1.12.0 (main) → immediately promoted → v2.0.0 (prod)</code>
Mature/Stable:	<code>v2.15.0 (main) → sits in QA</code>
	<code>v2.16.0 (main) → sits in QA</code>
	<code>v2.17.0 (main) → approved → v3.0.0 (prod)</code>

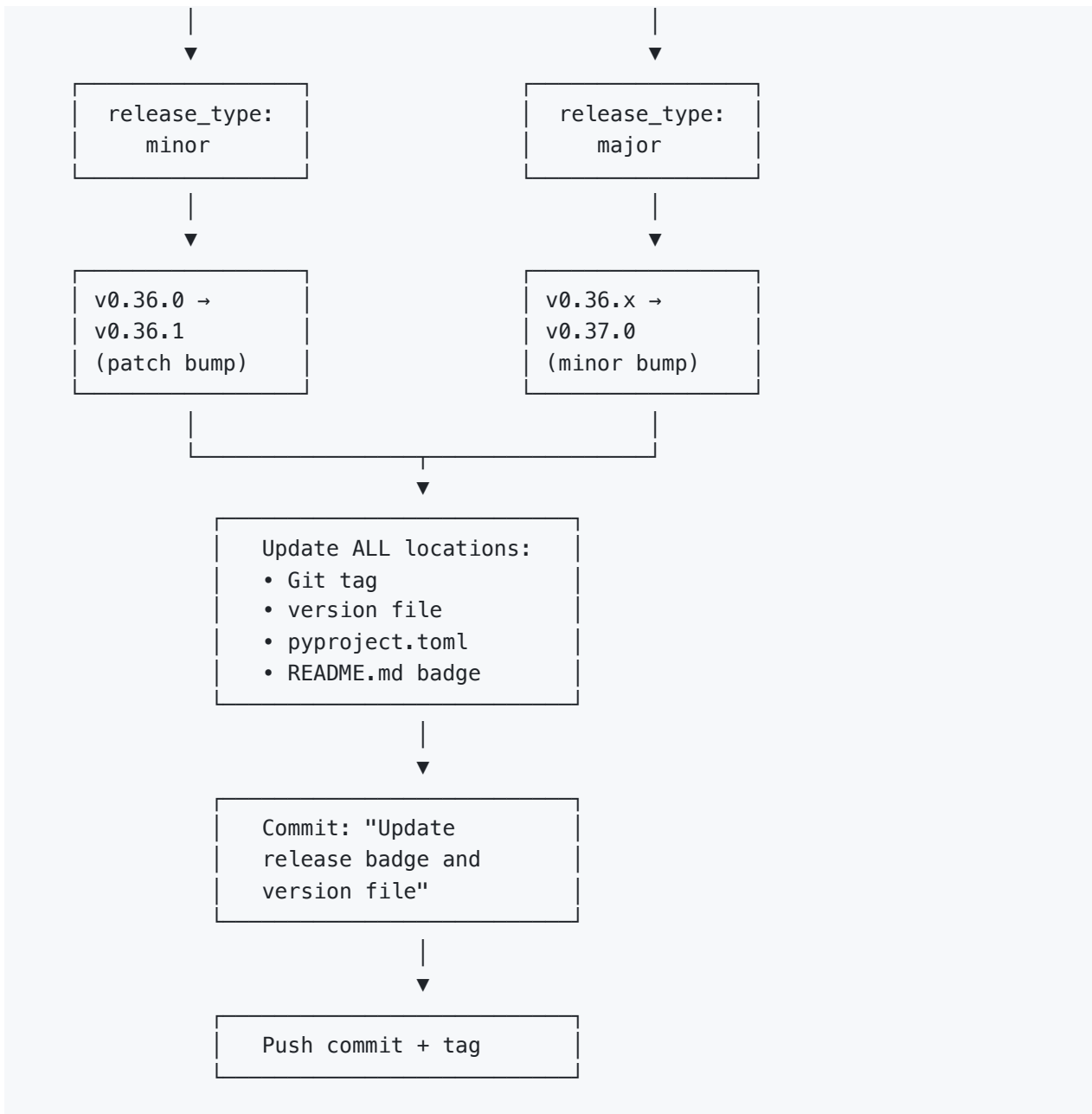
The `v1.0.0` milestone typically represents the first MVP—a service with real-world functionality backed by the full enterprise CI/CD pipeline this template provides.

Note: The rollout strategy (immediate, red-green, canary, gradual) is a separate concern not covered by this versioning system.

Version File Synchronization

The `git__increment-tag` GitHub Action maintains version consistency:





Version Semantics

The naming convention aligns with the three release types:

Release Type	Version Change	Trigger	Environment
minor	v0.36.0 → v0.36.1	Push to dev	Dev
major	v0.36.x → v0.37.0	Merge to main	QA
production	v0.x.x → v1.0.0	Manual (product owner)	Production

Remember: Production code == Major code. The production release is a business decision to ship already-tested code.

This produces a clear audit trail:

v0.37.0	← Merge to main (QA release)	
v0.36.1	← Dev work	

v0.36.0 ← Previous main merge
v0.35.3 ← Dev work
v0.35.2 ← Dev work
v0.35.1 ← Dev work
v0.35.0 ← Previous main merge

Git Graph Visualization



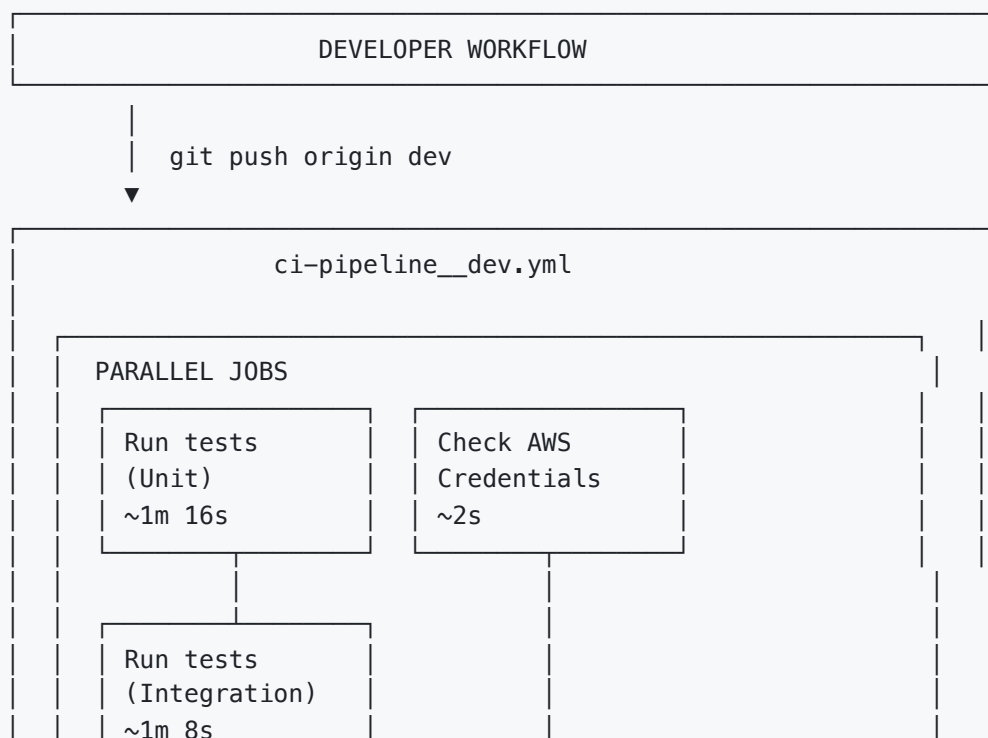
Every dot is a tagged, deployable state. Rollback to any point is trivial:

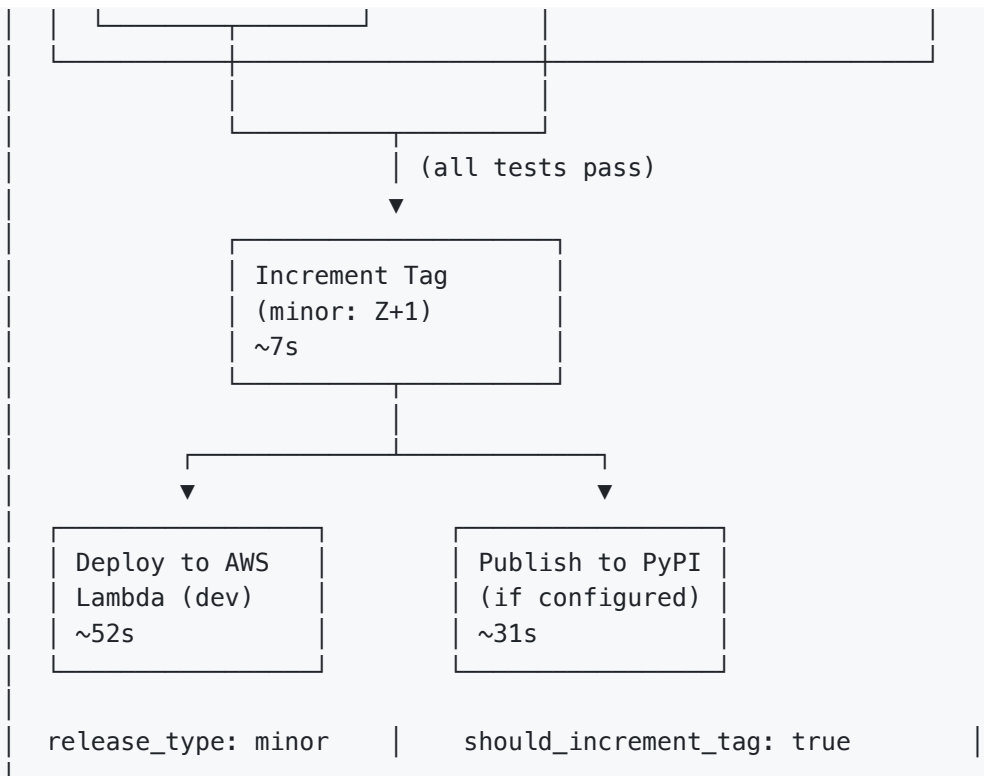
```
git checkout v0.36.1    # Exact state of that release
```

Part 4: Multi-Stage Deployment Pipeline

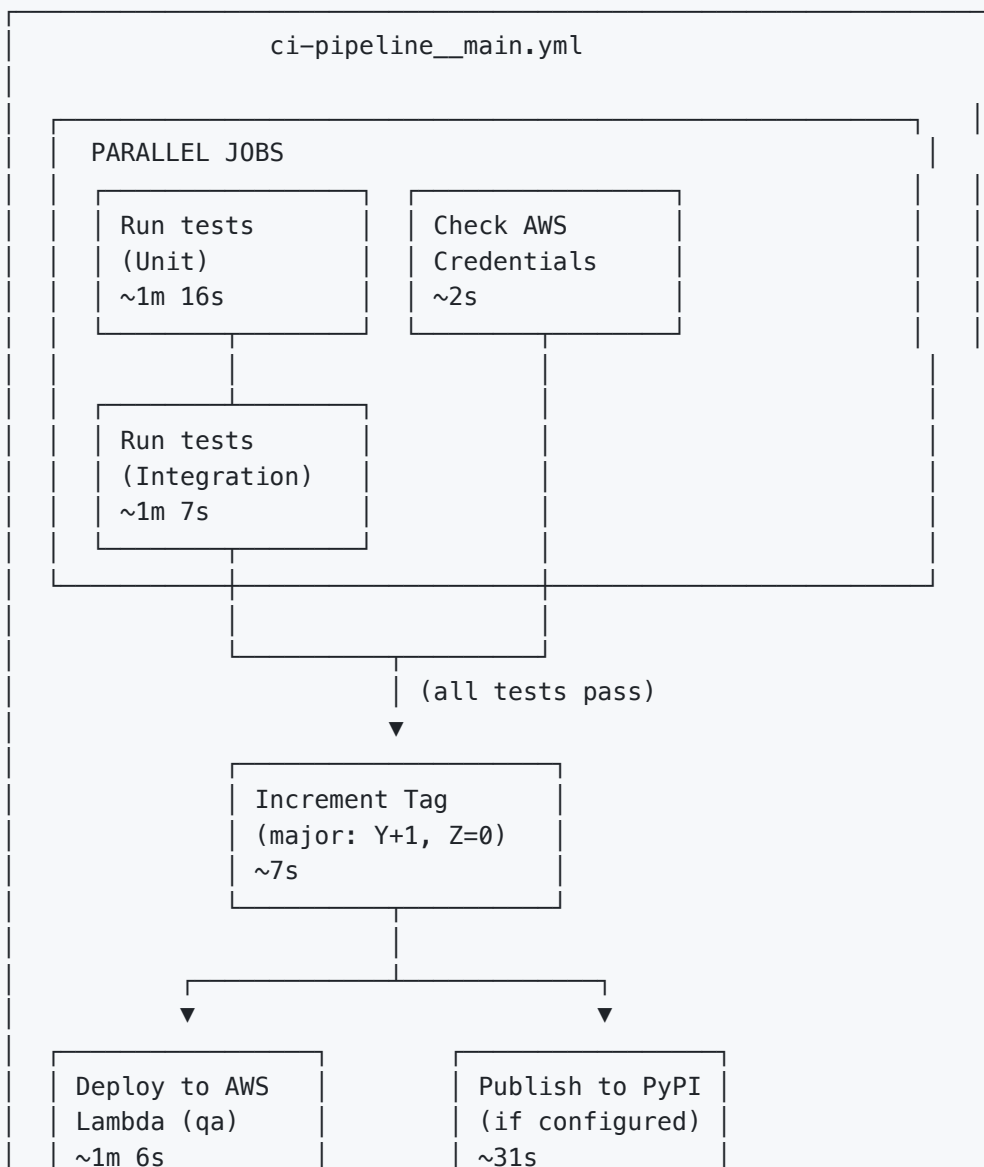
Pipeline Architecture

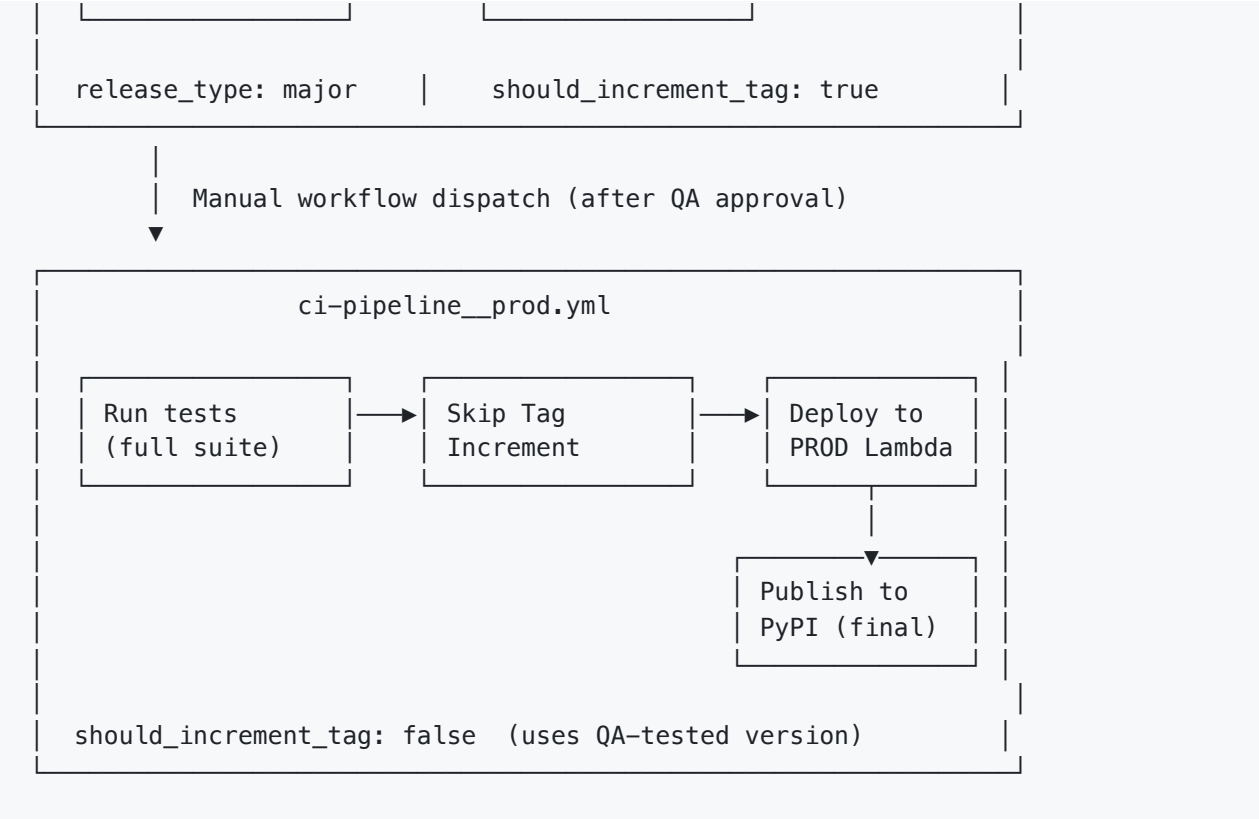
The CI/CD pipeline runs parallel test jobs, then gates deployment on both test success and credential availability:





git checkout main && git merge dev





PyPI Publishing Strategy

PyPI publishing is conditional and depends on project maturity:

Project Stage	Dev Branch	Main Branch	Production
Active development	Skip	✔ Publish	✔ Publish
Mature/Stable	Skip	Skip	✔ Publish only

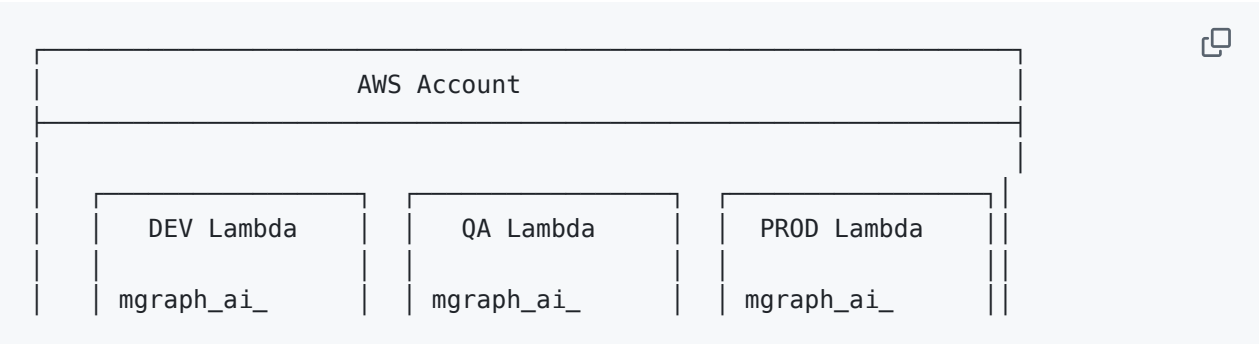
For actively developed projects: Publish to PyPI on every major release (main branch merge). This makes the latest QA-tested code available as a package.

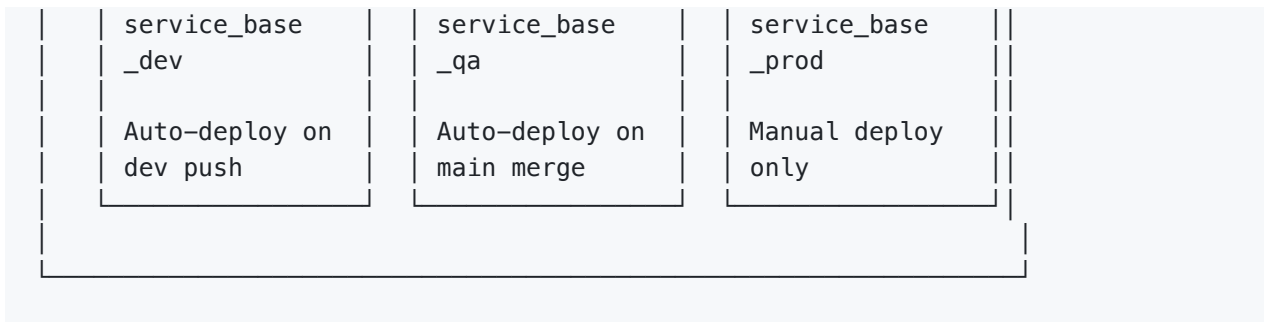
For mature projects: Only publish on production releases. This ensures only product-owner-approved versions reach PyPI.

The decision is configuration-driven—simply enable or disable the PyPI step in the relevant workflow file.

Environment Configuration

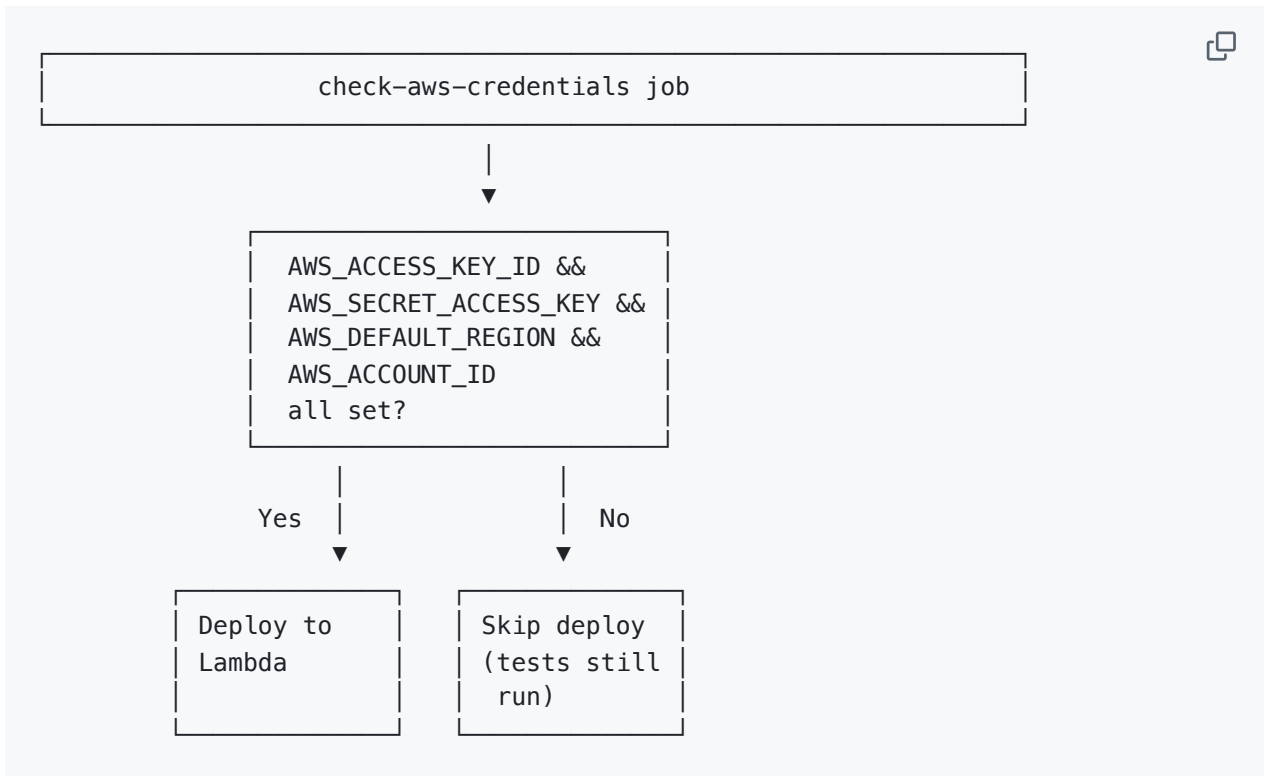
Each stage deploys to a separate Lambda function:





Conditional Deployment

The pipeline gracefully handles missing AWS credentials:

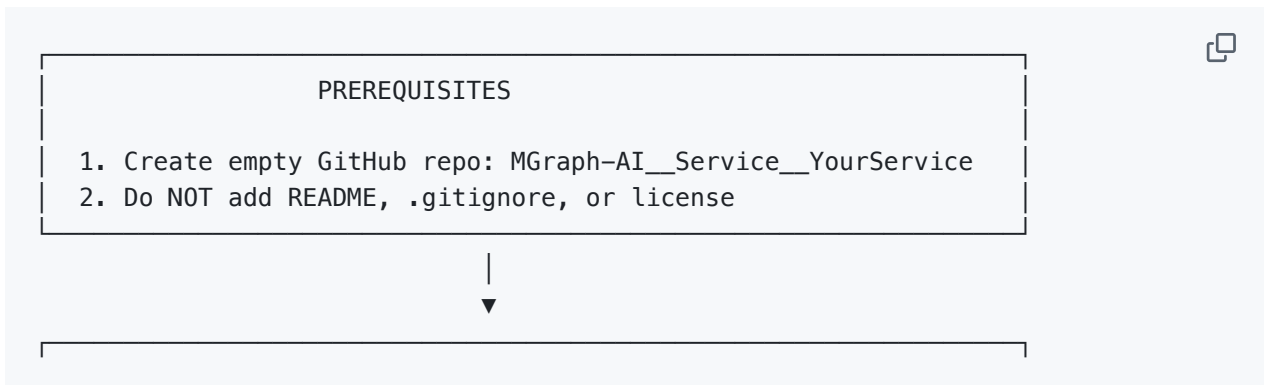


This allows the template to be used without AWS credentials during development.

Part 5: Service Bootstrap Process

Creating a New Service

The `setup-new-service-from-bare-github-repo.sh` script automates service creation:



```
./setup-new-service-from-bare-github-repo.sh  
MGraph-AI__Service__YourService
```



Step 1: Clone empty repository

```
git clone git@github.com:org/MGraph-AI__Service__YourService
```



Step 2: Create dev branch

```
git checkout -b dev
```



Step 3: Add template as remote

```
git remote add template [base_repo_url]
```



Step 4: Merge template content

```
git fetch template  
git merge template/main --allow-unrelated-histories
```



Step 5: Rename package directory

```
mv mgraph_ai_service_base → mgraph_ai_service_yourservice
```



Step 6: Global string replacements

- "MGraph-AI Service Base" → "MGraph AI Service YourService"
- "mgraph_ai_service_base" → "mgraph_ai_service_yourservice"
- "MGraph-AI__Service__Base" → "MGraph-AI__Service__YourService"

Applies to: *.py, *.md, *.yaml, *.toml, *.sh, *.txt, *.json



Step 7: Set initial version

```
echo "v0.1.0" > mgraph_ai_service_yourservice/version
```



Step 8: Create template tracking file

```
.template/VERSION with metadata
```



Step 9: Commit and push

```
git add . && git commit && git push -u origin dev
```

RESULT

- ✓ Repository: MGraph-AI__Service__YourService
- ✓ Package: mgraph_ai_service_yourservice
- ✓ CI/CD: Active on dev branch
- ✓ Version: v0.1.0
- ✓ Tests: Passing
- ✓ Ready for business logic

Post-Bootstrap: What You Get

After running the bootstrap script, your new repository has:

```
MGraph-AI__Service__YourService/
├── mgraph_ai_service_yourservice/  # Already renamed
│   ├── __init__.py                # Package name updated
│   ├── config.py                  # Service name updated
│   ├── version                    # Set to v0.1.0
│   ├── fast_api/                  # Ready for routes
│   └── ...
├── tests/                         # Tests passing
├── .github/workflows/             # CI active
├── pyproject.toml                 # Package name updated
├── README.md                      # Badges updated
└── .template/VERSION              # Tracks base version
```

The first CI run will:

1. Execute all unit tests (they pass)
2. Increment version to v0.1.1
3. Update all version files
4. Deploy to dev Lambda (if credentials configured)

Part 6: Key Configuration Files

pyproject.toml

```
[tool.poetry]
name          = "mgraph_ai_service_base"
```



```

version      = "v0.6.0"                # Auto-updated by CI
description  = "MGraph-AI__Service__Base"
authors      = ["Dinis Cruz <dinis.cruz@owasp.org>"]
license      = "Apache 2.0"

[tool.poetry.dependencies]
python       = "^3.12"
osbot-fast-api-serverless = "*"          # Core dependency

[build-system]
requires     = ["poetry-core>=1.0.0"]
build-backend = "poetry.core.masonry.api"

```

config.py

```

from mgraph_ai_service_base import package_name

SERVICE_NAME          = package_name
FAST_API__TITLE         = "MGraph-AI Service Base"
FAST_API__DESCRIPTION   = "Base template for MGraph-AI microservice"
LAMBDA_DEPENDENCIES__BASE__SERVICE = ['osbot-fast-api-serverless==v1.2.0']

```

Environment Variables

Variable	Purpose	Required
FAST_API__AUTH__API_KEY__NAME	Header name for API key	Yes
FAST_API__AUTH__API_KEY__VALUE	API key value	Yes
AWS_ACCESS_KEY_ID	AWS credentials	For deployment
AWS_SECRET_ACCESS_KEY	AWS credentials	For deployment
AWS_DEFAULT_REGION	AWS region	For deployment
AWS_ACCOUNT_ID	AWS account	For deployment

Part 7: Operational Runbook

Local Development

```

# Install dependencies
pip install -r requirements-test.txt
pip install -e .

# Set auth (required)
export FAST_API__AUTH__API_KEY__NAME="x-api-key"
export FAST_API__AUTH__API_KEY__VALUE="dev-key-123"

# Run server

```

```
uvicorn mgraph_ai_service_base.fast_api.lambda_handler:app \
    --reload --host 0.0.0.0 --port 10011

# Test endpoints
curl -H "x-api-key: dev-key-123" http://localhost:10011/info/health
curl -H "x-api-key: dev-key-123" http://localhost:10011/info/versions
```

Running Tests

```
# All unit tests
pytest tests/unit/

# With coverage
pytest tests/unit/ --cov=mgraph_ai_service_base

# Specific test
pytest tests/unit/fast_api/test_Service__Fast_API__client.py -v
```



Manual Deployment

```
# Deploy to specific environment
pytest tests/deploy_aws/test_Deploy__Service__to__dev.py
pytest tests/deploy_aws/test_Deploy__Service__to__qa.py
pytest tests/deploy_aws/test_Deploy__Service__to__prod.py
```



Rollback Procedure

```
# Identify target version
git tag --list

# Option 1: Redeploy specific version
git checkout v0.35.0
pytest tests/deploy_aws/test_Deploy__Service__to__prod.py

# Option 2: Revert and push (triggers normal pipeline)
git revert HEAD
git push origin main
```



Conclusion

MGraph-AI Service Base represents a philosophy: **infrastructure is not an afterthought**. By front-loading CI/CD, testing, versioning, and deployment into a reusable template, teams can focus on what matters—building features—while maintaining production-grade operational practices from day one.

The combination of automated versioning, multi-stage deployment, and the bootstrap script creates a reproducible, auditable, and scalable foundation for microservice development.

Appendix: Quick Reference

Version Bump Rules

Release Type	Branch	Version Change	Trigger
minor	dev push	v0.36.0 → v0.36.1	Automatic (requires tests to pass)
major	main merge	v0.36.x → v0.37.0	Automatic
production	manual	v0.x.x → v1.0.0	Product owner decision

Key principles:

- No "patch" concept—all changes flow through dev → main → production
- Production release == exact same code as the major release
- If tests pass, code is ready for production
- Version numbers can grow large (this is normal and expected)

Files Updated on Version Bump

1. {package_name}/version
2. pyproject.toml
3. README.md (badge)
4. Git tag

Endpoint Reference

Endpoint	Method	Description
/info/health	GET	Health check
/info/versions	GET	Dependency versions
/info/status	GET	Service status